Publish/Subscribe State-of-the-Art

Alexandros Antonov, Evangelos Kolyvas, and Spyros Voulgaris

Department of Informatics
Athens University of Economics and Business
Athens, Greece

Project: Pub/Sub Framework for the Cardano Ecosystem
Deliverable D1 – August 2024

Contents

Chapter 1

Introduction

As a foundation for the design and implementation of a messaging system based on the publish-subscribe (pub/sub) paradigm, it is crucial to perform a comprehensive literature review of the pub/sub paradigm. This review should identify the various components that make up a pub/sub system, explore the parameters involved and their impacts, and analyze and compare different system designs that have been popular and well-studied in the past.

Grasping the core principles of the pub/sub communication paradigm and understanding the distinctions between different approaches in the literature will facilitate the design of a pub/sub-based messaging system that meets the needs of the Cardano ecosystem. This document will be instrumental in guiding the design and implementation of the pub/sub system, providing directions for implementation details and assisting with parameterization.

Chapter 2

The Publish-Subscribe Communication Paradigm

The pub/sub communication paradigm is a fundamental architectural model used to establish communication channels between a message/event producer (publisher) and multiple receivers (subscribers) who wish to receive these messages or events.

The channels are designed so that neither the publisher nor the subscribers need to be concerned with questions such as "Where should I send the event I want to propagate?", "Did my event arrive at its destinations?", or "Should I check the channel for newly arrived events?". The pub/sub infrastructure handles the entire event transmission process, enabling a simple "send and forget" behavior. This approach reduces the dependencies between senders and receivers, significantly enhancing scalability and efficiency in distributed systems.

Each channel is referred to as a *topic*, and each topic has its own semantics. In some cases, a topic may have a single, static publisher, as is common in multimedia streaming. In other cases, the publishers may vary, with hosts potentially assuming both publisher and subscriber roles, as seen in chat applications. This flexibility allows the pub/sub model to adapt to a wide range of use cases and dynamic environments.

The substance of pub/sub infrastructure, should provide mechanisms that facilitate event exchange between the publishers and the subscribers, enabling decoupling along three key dimensions [1]:

- **Time decoupling:** Publishers and subscribers do not need to be online simultaneously. This functionality is often achieved through an intermediary entity that stores events and forwards them to consumers on demand. In distributed environments, advanced communication primitives can be implemented to ensure communication appears asynchronous and anonymous to the application, eliminating the need for an intermediary.
- **Space decoupling:** Publishers do not need to know the network ad-

dresses of subscribers to disseminate events. Specifically, publishers are unaware of the number of subscribers involved in the interaction and do not require direct references to them.

- **Synchronization decoupling:** A publisher does not need to wait for the subscriber to respond in order to continue its workflow. Specifically, the production and consumption of events do not happen in the main flow of control of the publishers and subscribers and do not, therefore, happen synchronously.

According to [2], pub/sub is essentially a middleware that stores events, and matches the events against subscriptions in order to forward them to specific subscribers. Essentially, pub/sub removes the explicit dependencies between a publisher and a subscriber, reducing the need for coordination, and adapting the communication to the distributed nature of applications.

In addition to the aforementioned design aspects, a pub/sub implementation may provide other desirable properties, such as event reliability, and security.

- **Reliability:** In pub/sub systems, reliability refers to the assurance that messages are delivered consistently, even in the presence of failures or network issues. This includes mechanisms for retrying failed deliveries, confirming the receipt of messages, and maintaining high availability of the pub/sub infrastructure. Reliable systems are essential for applications that require guaranteed delivery and consistency of information.

- **Security:** In the context of pub/sub systems, security involves protecting the communication channels, ensuring that only authorized publishers can send events and only authorized subscribers can receive them. This includes implementing encryption for data in transit, authentication mechanisms to verify identities, and access control policies to manage permissions. Security measures help prevent unauthorized access, data breaches, and other malicious activities.

There are three general approaches to building pub/sub systems when it comes to their structure:

1. **Client/Server:** This is a centralized approach where all events pass through one or more centralized dedicated servers to be sent individually to all subscribers through direct point-to-point connections. Although this method delivers events over one-hop connections and might suffice for negligibly small sets of subscribers, it is inherently non-scalable. When the load exceeds the combined resources of the servers, additional resources must be rented in order to handle the increased resource demand.

2. **Broker-Based:** This approach utilizes dedicated nodes, often servers, which form a network overlay that constitutes the backbone of event dissemination. While being more scalable than the Client/Server approach, it requires more than one hop for event dissemination. The use of brokers helps distribute the load more evenly across the network, improving scalability and fault tolerance.

3. **Distributed:** In this approach, there is no central entity. Instead, the messaging medium comprises the hosts that participate in the event exchange. Publishers and subscribers interconnect to form a network overlay through which events are disseminated. Additional pub/sub functionalities are implemented in a distributed fashion. For example, in the Gossipsub protocol [3] each host stores the events it receives and retransmits them on demand when a host has missed specific events, creating a layer of persistence over the pub/sub participants. The distributed approach is considered the most scalable, as participants act both as clients, who utilize the pub/sub service, and servers, who offer their resources to facilitate the service.

Another dimension that differentiates various pub/sub approaches is the degree of expressiveness that clients can use when selecting the events they want to receive. In this regard, pub/sub systems can be categorized into two main types: *topic-based* and *content-based* systems.

In topic-based pub/sub systems, messages are classified into topics, and subscribers express interest in one or more topics. Messages are delivered to subscribers who have subscribed to the corresponding topics. This class of systems is often utilized in the context of message exchange, data streaming, logging, and monitoring applications. Popular real-world systems which employ a model similar to the topic-based pub/sub paradigm include: BitTorrent (for file distribution), IPFS (for content addressing and distribution), and the Ethereum Blockchain (for the dissemination of consensus messages).

Content-based pub/sub systems, on the other hand, utilize subscription predicates that allow subscribers to filter the events they receive, keeping only events that contain fields with specific values or a range of values. For example, in the context of a stock trading application, a user may want to receive events about stock trades only if the stock price falls within a specific range or if the trade volume exceeds a certain threshold. Real-time systems in this case include: Google Cloud Pub/Sub (selective delivery of messages based on the content of the events, supporting applications that require fine-grained control over the events they process), Apache Flink (real-time filtering and transformation of data streams), and Heron (real-time monitoring and analysis of social media activity).

Content-based pub/sub offers greater flexibility in message routing, as it allows fine-grained filtering based on message content. Topic-based pub/sub, on the other hand, offers simplicity and ease of understanding, as the routing is based on predefined topics. This model is straightforward for both publishers and subscribers since they only need to know the topics of interest. Topic-based pub/sub often provides better performance and scalability due to simpler matching algorithms, as messages are routed based on topics rather than evaluating complex routes based on content predicates. This simplicity reduces computational overhead and can handle a higher volume of messages more efficiently.

The use cases of the Cardano Blockchain require an alternative messaging channel, which is better suited to topic-based pub/sub systems. Topic-based

pub/sub systems provide a straightforward and scalable solution for efficient and effective communication within the network.

Chapter 3

Implementing a Pub/Sub System

The diversity of existing pub/sub approaches illustrates that there is no one-size-fits-all solution when it comes to designing and implementing a pub/sub system. Instead, a group-based messaging solution should be tailor-made or carefully adapted to meet the specific needs of each individual system.

In this chapter, we will elaborate on the various pub/sub designs we found in the literature, detailing their characteristics, advantages, and potential drawbacks.

3.1 Application-Level Multicast Approaches

Application-Level Multicast (ALM) approaches share similar semantics with distributed topic-based pub/sub systems and often fall under the same category. Although the difference in semantics may not be immediately apparent, our study revealed that application-level multicast designs are primarily focused on creating a separate, highly efficient network overlay for each topic, referred to as a *multicast group* in application-level multicast terminology.

The typical approach involves establishing a control structure with a connected graph among all nodes and constructing a multicast tree overlay on top of it. This tree-based structure facilitates rapid message dissemination and eliminates the transmission of redundant messages.

However, these approaches tend to be more resource-intensive compared to conventional pub/sub approaches because they require a larger number of messages to construct and maintain the overlay. For instance, approaches like Scribe [4] and Bayeux [5] use Distributed Hash Tables (DHTs) as their control structure. This approach (using DHT), is resource-intensive because each node must maintain an up-to-date routing table. Moreover, refining the multicast tree incurs significant costs, and there is a single point of failure at the multicast tree root.

Use cases for ALM approaches are time-critical applications such as live video streaming, video conferencing, and real-time online gaming. The following are some popular ALM approaches derived from the literature.

3.1.1 Scribe & Bayeux

Scribe and Bayeux are two approaches that support multiple individual topics by building multicast trees on top of Pastry and Tapestry DHTs, respectively. Each approach employs its respective DHT as the control structure. The multicast tree for each topic is constructed by incorporating DHT routes that include the topic participants.

From the two approaches, Scribe is superior, for the three following reasons:

1. **Messaging Costs:** Bayeux introduces additional messaging costs as subscriptions require a request message to travel from the subscriber to the root of the multicast tree and then back again from the root to the subscriber via a different route. Scribe on the other hand facilitates a clever subscription mechanism which requires a subscription to traverse at most one DHT path from a subscriber to a dedicated rendezvous-point.

2. **Load Management:** Scribe offers better load management by distributing membership management across the nodes of the multicast tree. In contrast, Bayeux stores the entire state of subscribers at the root, which can lead to a single point of failure and increased load on the root node.

3. **Multicast Tree Construction:** Both Pastry (on which Scribe is built) and Tapestry (on which Bayeux is built) exploit network locality in a similar manner. With each successive hop taken within the overlay network from the source towards the destination, the message traverses an exponentially increasing distance in the proximity space. Scribe builds the multicast tree from the subscribers towards the root, creating numerous "short links" near the subscribers. Bayeux, on the other hand, constructs the multicast tree from the root towards the subscribers, resulting in fewer "short links" near the root.

Additionally, Scribe provides the following desirable properties, derived from the Pastry DHT:

- **Scalable:** Pastry enables routing through a log(N) number of steps (where N is the number of nodes), and each node's neighbor map contains log(N) entries. The actual (physical) traveling distance is proportional to the traveling distance in Pastry.

- **Fault tolerance:** Each entry in the neighbor map includes one primary neighbor and two alternative neighbors to handle unexpected node failures.

Nonetheless, such approaches suffer from a single point of failure in the form of multicast tree roots. Additionally, message dissemination for a group may be facilitated by nodes that are not part of the specific group, which may lack the incentives to participate in this process.

3.1.2 Zigzag

Zigzag [6] is a single-source Application-Level Multicast (ALM) approach designed to optimize both the degree of each node and the height of the multicast tree. It achieves a node degree of $O(k^2)$ and a tree height of $O(log_k N)$, where k is a constant and N is the number of nodes.

Nodes in Zigzag are organized into H layers, each layer consisting of a number of clusters, forming an administrative hierarchy that reflects the logical relationships between nodes. This clustering helps manage the complexity and scalability of the network. Each cluster operates under a leader node, which coordinates communication within the cluster and between clusters.

A multicast tree is constructed on top of this hierarchical structure, reflecting the physical relationships among nodes. Zigzag is engineered to meet several critical performance goals: short end-to-end delay, low control overhead, efficient join and failure recovery, and low maintenance overhead.

3.1.3 Nice

NICE [7] represents another layered multicast approach, where each layer is divided into clusters. Each cluster is led by the node that has the minimum maximum distance to all other nodes within the same cluster, thus incorporating proximity into the design. This proximity-based clustering is leveraged to construct balanced multicast trees for efficient data dissemination across all nodes.

3.2 Distributed Pub/Sub Approaches

In contrast to ALM, which focuses on constructing a single, efficient topology per topic, conventional distributed pub/sub system designs prioritize scalability as the number of nodes and topics grows.

To solve these problems, pure pub/sub systems take more straightforward approaches when constructing topic overlays, such as providing each node with a small set of links to random other nodes, leading to a random graph per topic. The selection of links is done smartly to ensure that links can be utilized in multiple topics, allowing a smaller growth in the number of links each node must maintain as the number of topics each node is subscribed to increases.

Specifically, as a system grows, the number of nodes and topics increases, leading to an increased number of nodes per topic and an increased number of topics each node participates in. For ALM approaches, in large systems, possibly consisting of millions of nodes, high overhead is introduced for each node, as they must store membership data for each individual topic multicast tree. Additionally, the complexity of the multicast-tree membership management grows exponentially as the number of nodes in the system increases.

Use cases for pure pub/sub system designs include chat applications, social media notifications, and IoT Systems which require additional scalability efforts

as they involve millions of devices working concurrently.

Some popular application-level multicast approaches derived from the literature follow.

3.2.1 Gossipsub

Gossipsub [3] is a distributed pub/sub protocol that facilitates the establishment of topics by enforcing each node to keep a small constant number of random links (approximately eight) per topic. The novelty of Gossipsub lies in its reactive approach to tackling malicious behavior by assessing neighbor nodes through a scoring function. This protocol is used in Ethereum and Filecoin to enable scalable message dissemination, particularly for messages involved in their consensus mechanisms.

Gossipsub is a push-pull, distributed pub/sub protocol consisting of three main components:

1. **Mesh Networks:** Nodes maintain distinct bidirectional links (approximately eight per node) for each pub/sub topic. These links form meshes utilized for rapid push-based dissemination of messages.

2. **Gossip Protocol:** Periodically, each node gossips with nodes from its subscribed topics, other than the mesh nodes, to share any missed messages. This ensures message propagation even if a node was temporarily disconnected or missed the mesh dissemination of a message.

3. **Peer Scoring:** Each node maintains individual scores for connected peers, evaluating their positive and negative behavior. Nodes retain links only to well-behaved peers, ensuring network reliability and resilience against malicious behavior.

The number of peers a node connects with is determined by the amplification factor, D. If a node's connections surpass D_{high}, slightly above D, or drop below D_{low}, slightly below D, the node will execute a *PRUNE* or *GRAFT* operation to adjust the number of connections accordingly.

The trade-off between traffic redundancy and reduced propagation latency is adjusted through the parameter D. A higher D results in transmitting a message to more neighbors for each dissemination hop, leading to faster propagation but also increasing dissemination overhead.

In figure 3.1, an example of a Gossipsub network for a single topic is shown. The bold lines correspond to mesh connections, forming the mesh network. When a message is initially published, it is rapidly transmitted via this mesh network. The faint lines correspond to the gossip connections, which are utilized for missed message re-transmission.
The scoring function that evaluates peer behavior is the following:

$$Score(peer) = TC \sum_{n=1}^{4} w_n(t_i) \times P_n(t_i)) + w_5 \times P_5 + w_6 \times P_6$$

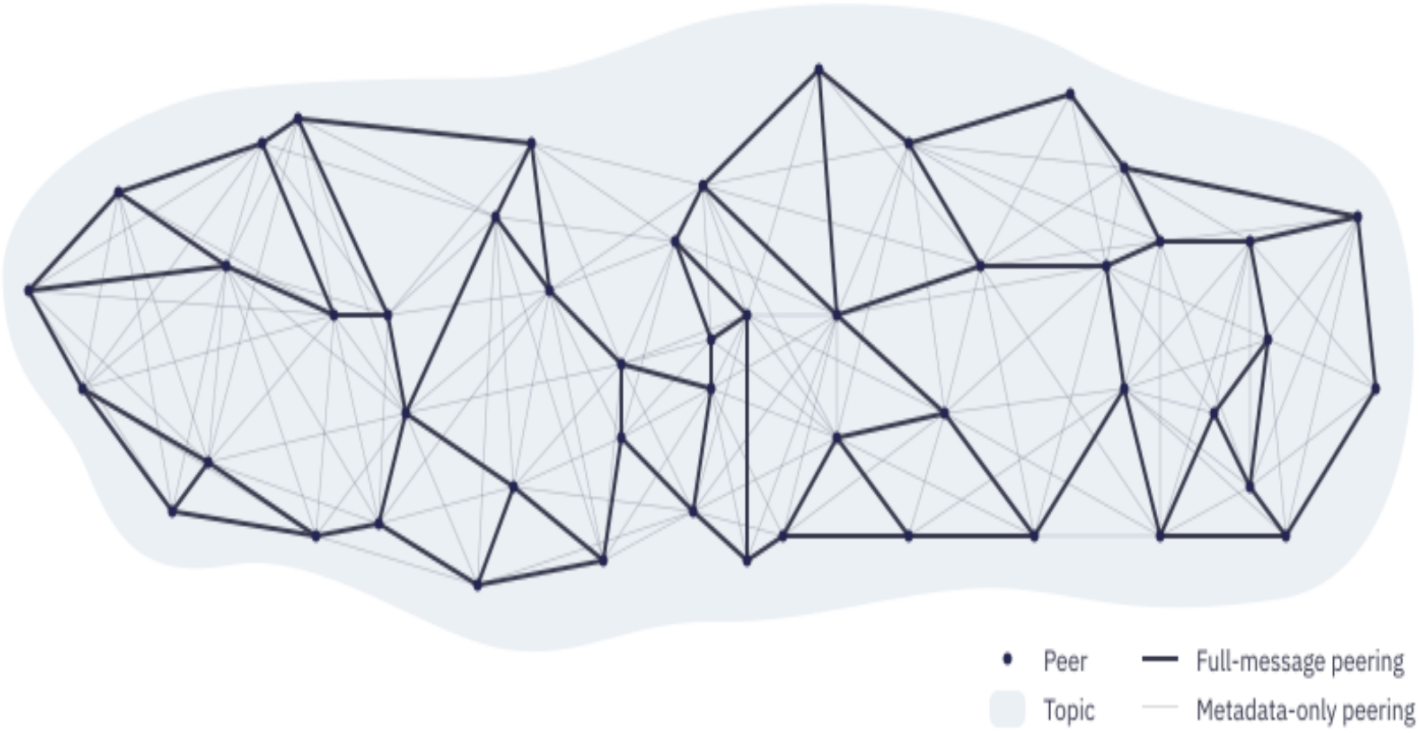

Figure 3.1: The network overlay for a single topic is created by the Gossipsub protocol. The overlay consists of a mesh network for fast event dissemination and a metadata network used to disseminate messages that nodes have missed.

The parameters in the function correspond to the time the neighbor peer has participated in the mesh, the number of times the peer is first to deliver a message to the node, the message delivery rate, message delivery failures, the number of invalid messages received from the peer, an application specific score, and an IP address collocation factor. Each value gets multiplied by a positive or negative weight factor. A higher peer score increases the likelihood of that peer being selected as a neighbor.

3.2.2 Poldercast

Poldercast [8] is a scalable pub/sub solution that provides 100% topic hit ratio in the absence of churn, and maintains a good hit ratio even when churn exists.

All subscribers of a topic T are connected through a ring topology, which guarantees 100% message delivery when there is no churn. In addition to the ring topology, random links are established between nodes within a topic, creating shortcuts across the linear paths of the ring topology and thus improving

dissemination speed. Additionally, these random links offer robustness in the form of redundancy and an improved hit ratio under churn. Essentially, random links help maintain each topic topology as a single connected component, specifically when nodes disconnect and topic rings break into multiple disconnected components.
Poldercast consists of the following three modules:

1. **The Ring module:** This module assigns each node its successor and predecessor, establishing a ring topology for each topic.

2. **Vicinity module:** This module provides a node with a set of links to random subscribers for each of the topics the node is subscribed to, based on a proximity function. The proximity function favors nodes based on two criteria. First, it favors peers that share as many common topics with the node in question as possible. Essentially, each node strives to reduce the number of links it maintains, by reusing the same links for multiple topics. Second, it favors peers of under-represented topics, ensuring that subscribers of non-popular topics manage to get well connected with each other.

3. **Cyclon module:** Provides a set of links to nodes chosen uniformly at random, from all network nodes. These links are used for keeping all subscribers connected in a single component, to support the Ring and Vicinity modules, and to improve load balancing.

In figure 3.2, a Poldercast topology in the presence of churn is presented. It is obvious that the number of links to random nodes make up for the gaps formed in the ring topology due to churn. This helps an event originating at some source to reach all subscribers of the topic with very high probability.

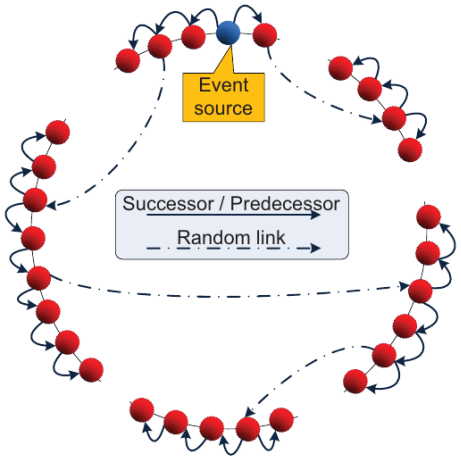

Figure 3.2: A network overlay, constructed by the Poldercast protocol. It is demonstrated that even in the presence of churn, events are successfully disseminated through the random links of the overlay.

3.2.3 Spidercast

Spidercast is a pub/sub protocol that forms *topic-connected* overlays for each individual topic. The property of topic-connectivity ensures that only the subscribers of a topic participate in the transmission of messages for that specific topic. This property is crucial in environments where nodes resemble rational users, as each user is highly motivated to participate in the timely dissemination of messages for the topics they are subscribed to, but should not be expected to engage in the dissemination of messages for other topics that do not concern them.

Other key features of Spidercast include:

- **Good Topic Connectivity:** The graph of each topic is dense, ensuring that disconnections of individual nodes do not result in multiple disconnected components within the overlay graph.
- **Scalable Average Node Degree:** By maintaining a moderate average node degree, each node ensures sustainable maintenance costs, as each node must periodically verify that its neighbors are still online.
- **Scalable Topic Diameter:** This feature ensures that messages are disseminated quickly, regardless of the number of subscriber nodes.

In Spidercast, nodes select neighbors based on two heuristics:

1. **Choose neighbors with the most topics in common.** This approach helps each node to maintain a moderate average degree.
2. **Choose random neighbors that have at least one topic in common with the node.** The addition of links to random nodes enhances the overall connectivity of a topic.

Nodes aim to maintain an adapted target degree L by adding or removing neighbors if their number exceeds or falls below specific thresholds. Furthermore, nodes take into account the view length of their neighbors when deciding which neighbors to remove, ensuring that these changes do not necessitate additional adjustments by their neighbors.

3.2.4 Tera

Tera [9] does not focus on routing optimization within a topic but rather addresses the challenge of peers joining new topics and publishers disseminating events to specific topics without prior knowledge of any nodes in those topics. This process is known as *outer-cluster routing*. As the number of network nodes, subscriptions per node, topics, and event publication rates grow, this problem becomes increasingly complex.

Tera consists of the following key components:

- **Subscription Manager:** Responsible for tracking the topics to which a node is currently subscribed, issuing requests to acquire an access point for each topic. Additionally, this component is used to advertise the list of the current access points possessed by the node to a set of nodes obtained through a random peer-sampling service.

- **Event Management:** Handles event routing from the publisher of each event to an access point of the corresponding topic, and then, the dissemination of the event to all nodes of the topic.
- **Access Point Lookup:** Utilizes a distributed search algorithm based on random walks to obtain a list of access points for queried topics.
- **Partition Merging:** Resolves the issue of two nodes concurrently initializing the same topic, which leads to the topic acquiring two different topic IDs.

The most important of these components is the Access Point Lookup component. Periodically, each node advertises all the topics it participates in to random nodes obtained through the peer sampling service. The Access Point Lookup components of the receivers, use these advertisements to maintain a table with a uniform random sample of all active topics, considering an estimation of the popularity of each topic (the number of subscribers of the topic).

A lookup table is used to find entry points when joining a new topic, or when disseminating an event to a topic a node is not subscribed to. If a node for a topic cannot be found in the local lookup table, a random walk is performed to acquire entry points from other random nodes.

3.3 Centrally Managed Pub/Sub Approaches

For a comprehensive overview, we highlight three pub/sub approaches from the literature that do not rely on the distributed structure of node participants for topic message dissemination. Instead, these approaches use a single or a set of designated nodes to facilitate message dissemination. The three approaches we discuss are Apache Kafka, Dynatops [10], and Dynamoth [11], representing both client/server and broker-based models, as detailed in the section 2.

Based on our observations, there is a noticeable shift in objectives for these more centralized pub/sub approaches. In the previously mentioned decentralized approaches, the main objective is to build and maintain sufficiently connected, fast-traversable network overlays. These approaches are usually scalable by design; as node participation increases, more resources become available to take advantage of during overlay construction and message dissemination.

More centralized approaches, on the other hand, do not require sophisticated network overlays, as message dissemination is facilitated by a very small number of well-known nodes. The main problem with these approaches is that scalability does not derive from the design. Mechanisms must be implemented to allocate additional resources when needed, and precise load balancing should be conducted to distribute the load as equally as possible among the allocated nodes.

3.3.1 Apache Kafka

Apache Kafka is an open-source, production-level pub/sub system designed for scalability and fault tolerance. The system consists of multiple servers, all aware

of the registered topics. Kafka achieves efficient message transactions through the use of partitions. Each topic is divided into multiple partitions, with each partition being managed by a single server. This allows publishers and subscribers to send and receive messages in parallel, enabling Kafka to scale efficiently.

For fault tolerance, Kafka replicates messages across partitions. Each partition is replicated on multiple servers, with one server acting as the leader and the others as replicas. All data transactions are processed through the leader, while replicas maintain copies of the data. If a leader becomes unavailable, one of the replicas takes over, ensuring continued data availability and system reliability. The replication configuration is set on a per-topic basis through the replication factor, which defines the number of replicas.

To track missing messages, Kafka uses a sequential offset to order messages within each partition of a topic. When a node comes back online, it can compare the offset of the last message in the partition with its own local offset. This allows the node to identify and request any missing messages that need to be retrieved.

The described architecture is illustrated in Figure 3.3.

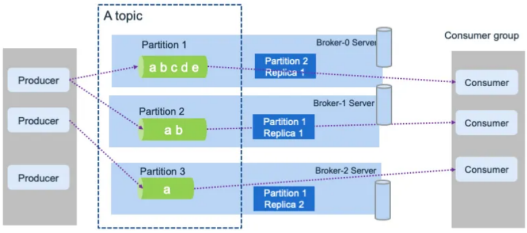

Figure 3.3: The architecture of a Kafka.

3.3.2 Dynamoth

Dynamoth employs a dynamic number of servers and sophisticated load balancing strategies to manage scenarios where topics undergo substantial changes in their subscriber and publisher groups. This system implements load balancing on two levels: channel-level rebalancing, which establishes various communication models to tackle extreme imbalances between publishers and subscribers, and system-level rebalancing, which focuses on distributing the load across servers.

The Dynamoth system comprises several servers, each equipped with a Local Load Analysis (LLA) module. These modules analyze traffic locally and send chunks of analysis data to a single external LoadBalancer, which provides plans based on the load of each server. Notably, servers in Dynamoth do not need to communicate with each other to synchronize topic subscriptions.

Given a topic c, and a number of active servers facilitating pub/sub for c, when an excessive disproportion between publishers and subscribers exists, one of the following two strategies is applied:

- **All-subscriber replication:** In this strategy, each subscriber sends its subscription to all servers, while publishers publish events by sending their messages to one randomly chosen server. This strategy requires fewer messages in the first hop of the dissemination, making it suitable for scenarios with a very high number of publishers.
- **All-publisher replication:** In this strategy, each subscriber establishes a subscription with only one of the servers responsible for topic c, and each server publishes events to all subscribers of c. Consequently, each subscriber receives an event only once, making this strategy suitable for scenarios with a very high number of subscribers.

Based on the load observed for each individual server, the following two plans may be issued:

- **High load on a server:** The server shares its load with other servers responsible for the same topic, or additional servers may be rented in order to facilitate the excessive load.
- **Low load on a server:** The server delegates its load to another server and shuts down to reduce the overall costs.

The design of a Dynamoth system is illustrated in figure 3.4.

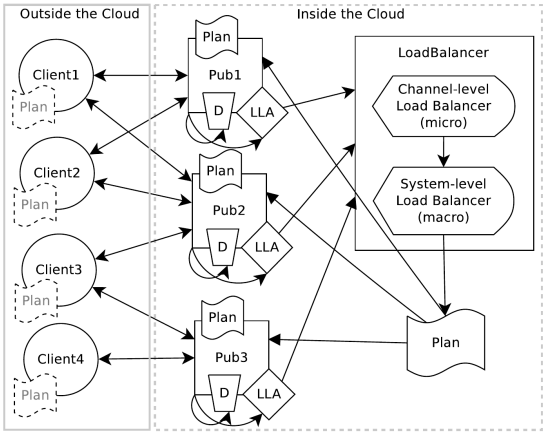

Figure 3.4: The design of a Dynamoth system.

An interesting argument presented in Dynamoth is that approaches using consistent hashing for load balancing have significant drawbacks. These approaches assume that channels are equally distributed across all virtual identifiers and that the load on each channel is equal, which may not be true in real-life environments. Dynamoth, on the other hand, strives to provide a more deterministic load balancing approach based on the observed load among the participating servers.

3.3.3 Dynatops

In Dynatops, a group of brokers collaborates on top of a DHT to ensure scalable message transmission, with a primary focus on reducing the number of hops needed for a message to reach its destination.

Dynamicity is addressed by a central entity, the Reconfiguration Manager, which has a comprehensive view of the network. This manager continuously runs a function to distribute subscribers to brokers based on two criteria: interest similarity and minimizing the number of subscriptions per broker. The Reconfiguration Manager monitors all active subscriptions and performs a cost-driven reconfiguration computation to find a balance between potential performance gains and reconfiguration costs, making necessary adjustments in the broker overlay.

Regarding user placement, an algorithm ensures that users with similar subscriptions are grouped and managed by the same set of brokers. The intuition behind this is that by clustering similar users, each broker facilitates topics that interest all or most of the users it hosts, making the subscription states of the brokers more stable and less dependent on individual users' subscription changes. Another benefit of this approach is that the number of brokers corresponding to each topic is minimized. This reduces the routing tree size, which directly improves the event dissemination speed for each topic.

3.4 Conclusions

The analysis of the publish-subscribe (pub/sub) paradigm reveals that its key strength lies in decoupling communication across time, space, and synchronization, making it highly scalable and adaptable for distributed systems like the Cardano Blockchain. Among the various pub/sub architectures—Client/Server, Broker-Based, and Distributed—the distributed model stands out for its scalability and resilience, aligning well with the decentralized nature of Cardano.

The report highlights various pub/sub approaches, each offering unique strengths in managing communication in a distributed fashion. By integrating elements from these approaches or designing new features tailored to Cardano's specific requirements, it is possible to develop a highly decentralized and scalable messaging system. Such a system would ensure reliable and efficient messaging between Cardano nodes, making it well-suited for real-time deployment.

Bibliography

[1] P. T. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, "The many faces of publish/subscribe," *ACM computing surveys (CSUR)*, vol. 35, no. 2, pp. 114–131, 2003.

[2] A.-M. Kermarrec and P. Triantafillou, "Xl peer-to-peer pub/sub systems," *ACM Computing Surveys (CSUR)*, vol. 46, no. 2, pp. 1–45, 2013.

[3] D. Vyzovitis, Y. Napora, D. McCormick, D. Dias, and Y. Psaras, "GossipSub: Attack-resilient message propagation in the Filecoin and ETH2.0 networks," *arXiv preprint arXiv:2007.02754*, 2020.

[4] M. Castro, P. Druschel, A.-M. Kermarrec, and A. I. Rowstron, "Scribe: A large-scale and decentralized application-level multicast infrastructure," *IEEE Journal on Selected Areas in communications*, vol. 20, no. 8, pp. 1489–1499, 2002.

[5] S. Q. Zhuang, B. Y. Zhao, A. D. Joseph, R. H. Katz, and J. D. Kubiatowicz, "Bayeux: An architecture for scalable and fault-tolerant wide-area data dissemination," in *Proceedings of the 11th international workshop on Network and operating systems support for digital audio and video*, 2001, pp. 11–20.

[6] D. A. Tran, K. A. Hua, and T. Do, "Zigzag: An efficient peer-to-peer scheme for media streaming," in *IEEE INFOCOM 2003. Twenty-second annual joint conference of the IEEE computer and communications societies (IEEE cat. No. 03CH37428)*, vol. 2. IEEE, 2003, pp. 1283–1292.

[7] S. Banerjee, B. Bhattacharjee, and C. Kommareddy, "Scalable application layer multicast," in *Proceedings of the 2002 conference on Applications, technologies, architectures, and protocols for computer communications*, 2002, pp. 205–217.

[8] V. Setty, M. Van Steen, R. Vitenberg, and S. Voulgaris, "Poldercast: Fast, robust, and scalable architecture for p2p topic-based pub/sub," in *Middleware 2012: ACM/IFIP/USENIX 13th International Middleware Conference, Montreal, QC, Canada, December 3-7, 2012. Proceedings 13*. Springer, 2012, pp. 271–291.

[9] R. Baldoni, R. Beraldi, V. Quema, L. Querzoni, and S. Tucci-Piergiovanni, "Tera: topic-based event routing for peer-to-peer architectures," in *Proceedings of the 2007 inaugural international conference on Distributed event-based systems*, 2007, pp. 2–13.

[10] Y. Zhao, K. Kim, and N. Venkatasubramanian, "Dynatops: A dynamic topic-based publish/subscribe architecture," in *Proceedings of the 7th ACM international conference on Distributed event-based systems*, 2013, pp. 75–86.

[11] T. Durieux and M. Monperrus, "Dynamoth: dynamic code synthesis for automatic program repair," in *Proceedings of the 11th International Workshop on Automation of Software Test*, 2016, pp. 85–91.